To finish **Stage 5: database documentation**, update your project's `README.md` file. This ensures anyone who clones your repository can get the app running in under five minutes without needing manual database configurations.

📝 Step 1: Update Your README.md

Open your project's `README.md` file. Replace or append to your existing notes with this clear structure:

markdown

```
# 📝 Task Management API (Database Version)

A persistent REST API built with a database backend.

## 🛠️ System Architecture Decisions

### Why SQLite?
- **Single File**: The entire database lives inside a single file on disk.
- **Zero Setup**: No separate server installation, configurations, or background processes
- **Data Persistence**: Data survives application crashes, server restarts, and system reb

### Database Location
- The data is stored in a file named `tasks.db` inside the project root folder.
- **Note**: This database file is created automatically on your first boot sequence. It is

---

## 🚀 Getting Started

Follow this one command to initialize and start the project locally:

### Option A: Node.js Lane
```bash
npm install && node server.js
```

Option B: Python Lane
```bash
pip install -r requirements.txt && uvicorn main:app --reload
```
```

```
---

## 🔍 Database Exploration (Stage 4)

### DB Browser Snapshot
Below is a screenshot of the database structure and entries open in DB Browser for SQLite:

![DB Browser Screenshot](./db_browser_screenshot.png)

### Hand-Executed SQL Query Example
- **Query Ran**: `SELECT * FROM tasks WHERE done = 1;`
- **Result Returned**: It filtered out all active tasks and returned rows matching only co
```

Use code with caution.

> 💡 **Tip:** Take a quick screenshot of your DB Browser view, save it as `db_browser_screenshot.png` in your project folder, and ensure the filename matches your Markdown image reference link.

🔍 Step 2: Critical Local Verification Check

Before you push your work, perform a quick local check to make sure it handles clean setups perfectly:

1. **Delete the database file:** Delete your local `tasks.db` file entirely.
2. **Boot the project:** Run your single documented startup command from the README.
3. **Verify the results:** Open your terminal client and execute:

bash

```
curl -i http://localhost:3000/tasks
# (Or port 8000 for Python users)
```

Use code with caution.

4. **Confirm success:** Verify that the server automatically re-created `tasks.db` and immediately returns your three seeded tasks with an HTTP **200 OK** status.

💾 Step 3: Git Commit and Global Deployment Push

Once your verification check is successful, stage your changes, run your final milestone commit, and push your entire codebase tree to your public GitHub repository:

bash

```
git add README.md db_browser_screenshot.png server.js main.py
git commit -m "Stage 5: database documentation"
git push origin main
```